

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6
Специальность ПО-13

Выполнил:
студент 3 курса
группы ПО-13
А.В. Король
Проверил:
А.Д.Кулик
01.04.2026

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`

Задание 1: Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart`

```
@pytest.fixture def  
empty_cart():  
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
```

```
def log_purchase(item): requests.post("https://example.com/log",  
    json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code): coupons =  
    {"SAVE10": 10, "HALF": 50}  
    if coupon_code in coupons:  
        cart.apply_discount(coupons[coupon_code])  
    else:  
        raise ValueError("Invalid coupon")
```

- Напишите тесты на `apply_coupon`

- Замокните словарь coupons с помощью monkeypatch или patch.dict

Выполнение:

Код программ:

test_cart.py

```
from unittest.mock import patch

import pytest

from shopping import Cart

@pytest.fixture
def empty_cart():
    return Cart()

def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10)

    assert len(empty_cart.items) == 1

def test_negative_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("Apple", -10)

def test_total(empty_cart):
    empty_cart.add_item("Apple", 10)
    empty_cart.add_item("Orange", 20)

    assert empty_cart.total() == 30

@pytest.mark.parametrize(
    "discount, result",
    [
        (0, 100),
        (50, 50),
        (100, 0),
    ],
)
def test_apply_discount(discount, result):
    cart = Cart()

    cart.add_item("Apple", 100)

    cart.apply_discount(discount)

    assert cart.items[0]["price"] == result
```

```

@pytest.mark.parametrize(
    "discount",
    [-1, 101],
)
def test_invalid_discount(discount):
    cart = Cart()

    cart.add_item("Apple", 100)

    with pytest.raises(ValueError):
        cart.apply_discount(discount)


def log_purchase(item):
    import requests

    requests.post(
        "https://example.com/log",
        json=item,
        timeout=10,
    )


@patch("requests.post")
def test_log_purchase(mock_post):
    item = {
        "name": "Apple",
        "price": 10,
    }

    log_purchase(item)

    mock_post.assert_called_once_with(
        "https://example.com/log",
        json=item,
        timeout=10,
    )

```

shopping.py

```

class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        if price < 0:
            raise ValueError("Price cannot be negative")

        self.items.append(
            {
                "name": name,
                "price": price,
            }
        )

```

```

def total(self):
    return sum(item["price"] for item in self.items)

def apply_discount(self, discount):
    if discount < 0 or discount > 100:
        raise ValueError("Invalid discount")

    for item in self.items:
        item["price"] -= item["price"] * discount / 100

```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы №1. Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Выполнение:

Код программы:

hamming_distance.py

```

def hamming_distance(str1, str2):
    if str1 is None and str2 is None:
        raise TypeError("Both strings are None")

    if str1 is None or str2 is None:
        return -1

    if len(str1) != len(str2):
        raise ValueError("Strings must have equal length")

    distance = 0

    for char1, char2 in zip(str1, str2):
        if char1 != char2:
            distance += 1

    return distance

```

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Выполнение:

Код программы:

test_hamming_distance.py

```

import pytest

from hamming_distance import hamming_distance

```

```

def test_none_none():
    with pytest.raises(TypeError):
        hamming_distance(None, None)

def test_first_none():
    assert hamming_distance(None, "abc") == -1

def test_second_none():
    assert hamming_distance("abc", None) == -1

def test_different_length():
    with pytest.raises(ValueError):
        hamming_distance("abc", "abcd")

def test_empty_strings():
    assert hamming_distance("", "") == 0

def test_equal_strings():
    assert hamming_distance("father", "father") == 0

def test_one_difference():
    assert hamming_distance("pip", "pop") == 1

def test_two_differences():
    assert hamming_distance("abcd", "abab") == 2

def test_hello_hallo():
    assert hamming_distance("hello", "hallo") == 1

def test_all_different():
    assert hamming_distance("abcd", "efgi") == 4

```

Рисунки с результатами работы программ

PS C:\Users\user\Documents\GitHub\spp_po13\reports\Korol\lab6> python -m pytest

===== test session starts =====

platform win32 -- Python 3.14.5, pytest-9.0.3, pluggy-1.6.0

rootdir: C:\Users\user\Documents\GitHub\spp_po13\reports\Korol\lab6

plugins: anyio-4.13.0

collected 19 items

src\test_cart.py

src\test_hamming_distance.py

[47%]

[100%]

===== 19 passed in 0.18s =====

Вывод: освоил приемы тестирования кода на примере использования пакета pytest